

State-Aware Performance Reproducibility in Stateful Serverless Applications

Aarti Sonigara

Department of Computer Science and Engineering
Uka Tarsadia University, Gujarat, India
aaratisoniga25@gmail.com

September 2026

Abstract

Stateful serverless computing enables persistent state to be maintained across serverless function executions, but the presence of persistent state introduces additional factors that may affect performance stability and measurement reproducibility. Existing serverless benchmarking studies commonly report average latency and related performance metrics, while comparatively less attention has been given to how state-related characteristics influence the reproducibility of repeated measurements.

This study investigates state-aware performance reproducibility in stateful serverless applications by systematically varying workload intensity, state size, and state-access patterns. A stateful serverless application using AWS Lambda and Amazon DynamoDB was implemented, and repeated experiments were conducted under different configurations. Performance variability was evaluated using mean latency, median latency, standard deviation, minimum and maximum latency, and coefficient of variation (CV).

The experimental results demonstrate that repeated executions of equivalent serverless configurations can exhibit substantial performance variability. Workload experiments showed that increasing the number of repetitions generally improved the stability of client-observed measurements. The access-pattern experiments revealed substantially higher variability for read-heavy access compared with write-heavy and mixed access patterns. The state-size experiments also showed differences in observed variability across the evaluated state configurations, although no simple monotonic relationship was observed.

The findings indicate that average latency alone is insufficient for characterizing performance reproducibility in stateful serverless systems. Instead, reproducibility-oriented benchmarking should explicitly consider state characteristics, workload conditions, access behavior, and measurement variability. This study provides an empirical foundation for state-

aware performance evaluation and motivates more comprehensive experimental methodologies for reproducible benchmarking of stateful serverless applications.

Keywords: Stateful Serverless Computing, Serverless Performance, Performance Reproducibility, Performance Variability, AWS Lambda, Amazon DynamoDB, State Size, State-Access Patterns, Cloud Computing

1 Introduction

Cloud computing has transformed the design and deployment of modern distributed applications by providing on-demand access to scalable computing, storage, and networking resources. Instead of maintaining dedicated physical infrastructure, organizations can dynamically provision computational resources according to application demand. This elasticity has enabled the development of distributed services that can accommodate highly variable workloads while reducing the operational effort associated with infrastructure management. However, as cloud-native applications have evolved, traditional virtual-machine and container-based deployment models have increasingly been complemented by more fine-grained execution models.

Serverless computing, particularly Function-as-a-Service (FaaS), represents one such evolution. In a serverless architecture, application functionality is divided into independently executable functions that are provisioned and scaled by the underlying platform in response to incoming events. Developers are therefore able to focus primarily on application logic rather than directly managing servers and infrastructure. The automatic scaling and pay-per-use characteristics of serverless computing make it attractive for event-driven, bursty, and highly variable workloads. Nevertheless, the abstraction of infrastructure does not eliminate the performance challenges associated with distributed execution. Function startup delays, resource contention, network communication, storage access, and workload fluctuations can all influence observed application performance.

Early serverless applications were predominantly designed using a stateless execution model, where individual function invocations did not depend on persistent local execution state. Although statelessness simplifies horizontal scaling and fault isolation, many real-world applications require information to be preserved across multiple function invocations. Examples include distributed workflows, transactional applications, collaborative services, session-oriented applications, stream processing, and applications requiring intermediate computation results. Stateful serverless computing addresses this limitation by allowing serverless functions to maintain or access persistent state through external or integrated state-management mechanisms.

The introduction of persistent state, however, changes the performance characteristics of serverless applications. A function invocation may need to retrieve, update, serialize, deserialize, synchronize, or migrate state before completing its computation. Consequently, application performance is influenced not only by

function execution time but also by the characteristics of state access and the environment in which the application executes. State size, state-access frequency, read/write intensity, workload concurrency, network conditions, storage behavior, and resource availability may introduce additional sources of latency and performance variation.

Performance variability is particularly important when serverless systems are evaluated experimentally. A performance measurement obtained from a single execution does not necessarily represent the behavior of the system under repeated executions. Even when the application code, workload, and configuration remain unchanged, subsequent executions may produce different latency, throughput, or resource-utilization measurements because of variations in the underlying execution environment and distributed system conditions. Such variability becomes especially significant for stateful serverless applications because state-related operations introduce additional dependencies between function execution and external state-management components.

Reproducibility is therefore an important requirement for experimental research in serverless computing. A performance experiment can be considered reproducible when independent executions performed under equivalent experimental conditions produce sufficiently consistent observations and lead to comparable conclusions. Poor reproducibility can make it difficult for researchers to determine whether an observed performance difference is caused by an actual system characteristic or by environmental and experimental variation. This issue can consequently affect the validity of comparisons between different serverless systems, configurations, and optimization techniques.

Existing research has extensively investigated serverless performance benchmarking, performance variability, state management, state placement, state migration, and execution optimization. Standardized benchmarking approaches have attempted to improve the comparability of serverless performance measurements, while other studies have investigated the causes of performance fluctuations in serverless environments. In parallel, research on stateful serverless computing has explored mechanisms for storing, accessing, placing, and migrating application state. These studies provide important foundations for understanding both serverless performance and stateful execution.

However, the interaction between state-related characteristics and the reproducibility of performance measurements remains comparatively less explored. In particular, it is not yet sufficiently clear how changes in state size, state-access patterns, workload intensity, and execution-environment conditions influence the stability of repeated performance measurements in stateful serverless applications. A performance evaluation methodology that treats state as an explicit experimental factor may therefore provide a more accurate understanding of performance variability and reproducibility in stateful serverless environments.

To address this research gap, this study investigates the reproducibility of performance measurements in stateful serverless applications using a state-aware experimental methodology. The study systematically varies selected state-related and workload-related parameters while repeatedly executing equivalent experimental configurations. Performance observations are collected using metrics

such as latency, tail latency, throughput, state-access overhead, standard deviation, coefficient of variation, and confidence intervals. The resulting measurements are analyzed to determine the extent to which state-related factors contribute to performance variability and measurement instability.

The primary objective of this research is not simply to measure the performance of a particular serverless platform, but to understand the conditions under which performance measurements of stateful serverless applications remain consistent and reproducible. By explicitly incorporating state characteristics into the experimental design, this research aims to provide a systematic basis for evaluating and reporting performance experiments in stateful serverless environments. The findings may assist researchers in designing more reliable experiments and in distinguishing genuine system-level performance effects from variability introduced by state, workload, or execution-environment conditions.

The main contributions of this research are therefore threefold: (1) a state-aware methodology for evaluating the reproducibility of performance measurements in stateful serverless applications; (2) an empirical analysis of the relationship between state-related factors, workload characteristics, and performance variability; and (3) a set of evidence-based recommendations for conducting and reporting reproducible performance experiments in stateful serverless environments.

2 Related Work and Literature Review

2.1 Serverless Performance Benchmarking

Serverless performance benchmarking has become an important area of research because the performance of Function-as-a-Service (FaaS) applications can vary across platforms, workloads, and execution environments. Researchers have proposed benchmarking methodologies to provide systematic and comparable measurements of serverless applications. These studies commonly evaluate metrics such as execution latency, throughput, cold-start overhead, resource utilization, and execution cost.

A major objective of existing benchmarking research is to improve the fairness and repeatability of comparisons between different serverless platforms. Standardized workloads and controlled experimental configurations can reduce differences caused by inconsistent testing procedures. However, serverless performance measurements may still be affected by factors outside the application logic, including resource allocation, scheduling, network conditions, platform-level contention, and variations between repeated executions.

Existing benchmarking approaches provide an important foundation for evaluating serverless systems, but most performance evaluation methodologies primarily focus on application execution characteristics and platform-level behavior. For stateful serverless applications, persistent state introduces additional experimental dimensions, such as state size, state-access frequency, read/write patterns, and state-management overhead. These factors can influence both

measured performance and the variability observed across repeated executions. Therefore, a benchmarking methodology that explicitly considers state-related characteristics is necessary for studying the reproducibility of performance measurements in stateful serverless applications.

2.2 Performance Variability and Reproducibility

Performance variability is an important characteristic of serverless computing environments because the observed performance of an application can change even when the application code and input workload remain unchanged. Unlike traditional dedicated environments, serverless applications execute on infrastructure that is dynamically managed by the cloud provider. Consequently, factors such as function initialization, resource allocation, scheduling decisions, platform-level contention, network conditions, and the distinction between cold and warm executions can contribute to variations in execution performance.

Several studies have investigated performance variability in serverless environments and have demonstrated that repeated executions may produce different latency and resource measurements. Average execution time alone may therefore be insufficient to characterize the actual behavior of a serverless application. Tail-latency measurements, including high-percentile latency, can reveal variations that may remain hidden when only mean performance is considered. Similarly, statistical measures such as standard deviation, coefficient of variation, and confidence intervals can provide additional information about the stability of experimental measurements.

Reproducibility is closely related to this problem. In experimental performance evaluation, reproducibility requires that an experiment conducted again under equivalent conditions produces sufficiently consistent observations and supports comparable conclusions. A lack of reproducibility can make it difficult to determine whether a reported performance difference represents a genuine characteristic of the evaluated system or is caused by temporary environmental conditions. This is particularly challenging in cloud environments where researchers generally do not control the complete underlying infrastructure.

Existing serverless benchmarking research has therefore emphasized the importance of standardized workloads, controlled configurations, repeated measurements, and clearly defined performance metrics. Such practices can improve the comparability of experimental results across different studies and platforms. However, reproducibility remains challenging because serverless execution environments are inherently dynamic and may introduce sources of variation that cannot be completely controlled by the researcher.

The reproducibility problem becomes more complex for stateful serverless applications. Persistent state introduces additional factors that can influence execution behavior, including state size, state-access frequency, read and write operations, serialization and deserialization, state-store latency, and state-management overhead. These factors may interact with workload characteristics and platform conditions, potentially increasing the variability of repeated performance measurements.

Therefore, evaluating reproducibility in stateful serverless applications requires more than simply repeating the same benchmark. The experimental methodology must explicitly account for state-related characteristics and examine how they influence the distribution and stability of performance measurements. This motivates a state-aware approach in which state configuration, workload conditions, execution environment, and repeated observations are considered together when evaluating performance reproducibility.

2.3 Stateful Serverless Computing

Stateful serverless computing has emerged as an extension of the traditional stateless serverless execution model to support applications that require information to persist across multiple function invocations. In a conventional stateless Function-as-a-Service architecture, each function invocation is generally treated as an independent execution. Although this model provides advantages for scalability and fault isolation, maintaining application state externally can introduce additional communication and coordination overhead. Stateful serverless approaches therefore aim to provide mechanisms through which functions can efficiently maintain, access, and coordinate persistent application state.

Research in stateful serverless computing has investigated different approaches for integrating persistent state with serverless execution. These approaches include external state stores, distributed in-memory storage, transactional state management, state-aware scheduling, state placement, and state migration. The primary objective of these systems is to reduce the overhead associated with accessing persistent state while maintaining the elasticity and scalability expected from serverless platforms.

State placement is an important aspect of stateful serverless execution. The location of application state relative to the function executing the computation can influence network latency, data-transfer overhead, and overall execution time. Researchers have therefore investigated techniques for placing state closer to computation and dynamically adapting state placement according to workload behavior. Similarly, state migration has been studied as a mechanism for moving application state when functions or workloads change their execution locations. Although migration can improve locality, it may also introduce additional data-transfer and coordination costs.

Another important research direction concerns the consistency and coordination of shared state. When multiple function instances concurrently access or modify the same state, the underlying system must provide an appropriate consistency model while avoiding excessive synchronization overhead. Different state-management mechanisms therefore make different trade-offs between consistency, performance, scalability, and fault tolerance. These trade-offs can directly influence the performance observed by applications using stateful serverless systems.

Performance evaluation is consequently an important component of stateful serverless research. Existing systems have commonly evaluated metrics such as function execution latency, state-access latency, throughput, scalability, com-

munication overhead, and the cost of state operations. Such evaluations demonstrate the performance characteristics of individual stateful serverless architectures and state-management techniques under selected workloads.

However, the performance of stateful serverless applications is not determined solely by the underlying state-management mechanism. Experimental factors such as state size, state-access frequency, read/write patterns, workload concurrency, execution locality, and environmental conditions can interact with one another and influence measured performance. As a result, measurements obtained from a particular experimental configuration may not necessarily remain stable across repeated executions.

This observation motivates a closer examination of stateful serverless performance from the perspective of reproducibility. While existing research has primarily focused on enabling efficient state management, reducing state-access overhead, improving state placement, and supporting state migration, comparatively less attention has been devoted to systematically quantifying how state-related characteristics affect the reproducibility of performance measurements. This distinction forms an important foundation for the state-aware performance reproducibility investigation presented in this research.

2.4 State Management and State-Related Performance

State management is a fundamental component of stateful serverless computing because serverless functions are typically short-lived and independently scalable, while many applications require information to persist across multiple invocations. A common approach is to maintain application state in external storage systems such as distributed key-value stores, databases, or other persistent storage services. Although this approach enables serverless functions to access shared state, it can introduce additional communication, synchronization, and data-access overhead. The latency of accessing externally managed state can therefore become an important component of the overall application execution time.

Several research efforts have proposed specialized mechanisms for improving state management in serverless environments. These systems investigate techniques such as local state caching, distributed state stores, transactional state management, and state-aware execution. For example, stateful serverless systems have explored integrating serverless execution with distributed key-value stores and providing local caching mechanisms to reduce the cost of repeatedly accessing remote state. Such approaches demonstrate that the organization and location of state can have a direct influence on application performance.

State placement is another important research direction. The physical or logical location of application state relative to the function executing the computation can influence data-transfer latency and network overhead. Function placement and state placement strategies therefore attempt to improve data locality by placing computation and frequently accessed state closer together. Recent research has also investigated dependency-aware placement for stateful serverless workflows, where the relationships between multiple functions and

their shared state are considered when making runtime placement decisions. These approaches demonstrate that state locality can significantly affect end-to-end workflow performance.

State migration has similarly received attention in stateful serverless systems. As workloads change or resources become imbalanced, moving functions or their associated state between execution locations can improve locality and resource utilization. However, migration itself introduces additional costs because state must be transferred and application consistency must be maintained during the transition. Recent work on stateful serverless workflows has investigated dynamic migration and state placement together, demonstrating that migration decisions can influence network traffic, latency, and overall workflow performance.

The size and access characteristics of application state also represent important performance factors. Larger state objects may require additional data-transfer, serialization, deserialization, and storage operations. Similarly, read-intensive, write-intensive, and mixed access patterns may produce different performance characteristics because they impose different demands on the underlying state-management mechanism. Experimental evaluations of stateful serverless systems have therefore considered state size, event frequency, state growth, and different state-management strategies when analyzing application performance.

Existing research has consequently established that state management is closely connected to the performance of stateful serverless applications. Studies have evaluated state access latency, throughput, data locality, network overhead, migration cost, scalability, and consistency. However, these performance evaluations are generally designed to determine whether a particular state-management mechanism or system provides better performance under selected experimental conditions.

An important distinction arises between measuring state-related performance and evaluating the reproducibility of those measurements. A system may demonstrate a particular latency or throughput under one experimental configuration, but repeated executions of the same configuration may produce different observations. Factors such as state size, state-access pattern, workload intensity, state-store behavior, network conditions, and execution-environment variability may contribute to these differences.

Therefore, state-related characteristics should not be treated only as implementation parameters when evaluating stateful serverless performance. They should also be considered as experimental factors that may influence the stability of performance measurements. A systematic investigation of the relationship between state characteristics and measurement variability can provide additional information beyond conventional performance benchmarking.

This observation motivates the focus of the present research. Rather than proposing another state-management, state-placement, or state-migration mechanism, this study investigates how state-related characteristics influence the reproducibility of performance measurements in stateful serverless applications. In particular, the study considers state size and state-access patterns together

with workload and execution conditions, and evaluates their influence on repeated performance measurements using statistical indicators. This perspective complements existing state-management research by examining not only how well a stateful serverless system performs, but also how consistently its measured performance can be reproduced under equivalent experimental conditions.

2.5 Research Gap

The review of existing research indicates that substantial progress has been made in the areas of serverless performance benchmarking, performance variability, reproducible experimentation, stateful serverless execution, and state management. Existing benchmarking studies have proposed standardized methodologies for evaluating serverless applications, while performance studies have investigated factors such as latency, cold-start behavior, workload characteristics, and execution-environment variability. Research on stateful serverless computing has additionally explored persistent state management, state placement, state migration, state locality, consistency, and state-access optimization.

However, the existing body of research primarily evaluates these aspects independently or focuses on improving the performance of a particular serverless architecture or state-management mechanism. Although stateful serverless systems have been evaluated using different workloads and state configurations, comparatively limited systematic attention has been given to the reproducibility of their performance measurements when state-related characteristics are explicitly varied.

In particular, there remains a need to understand how state size and state-access patterns influence the variability of repeated performance measurements under equivalent experimental conditions. Furthermore, the interaction between state characteristics, workload intensity, and execution-environment conditions has not been sufficiently characterized from a reproducibility perspective. Conventional benchmarking can report average latency, tail latency, throughput, or execution cost, but these measurements alone do not establish whether the reported observations remain stable when the same experiment is repeated.

This research therefore addresses the gap between conventional performance evaluation and state-aware performance reproducibility. Instead of proposing another state-management, state-placement, or state-migration mechanism, the proposed study treats state characteristics as explicit experimental factors and investigates their influence on the stability of performance measurements. The study will systematically vary state size, state-access patterns, workload conditions, and selected execution-environment factors while repeatedly executing equivalent configurations.

The research will use statistical measures, including standard deviation, coefficient of variation, confidence intervals, and latency distributions, to quantify performance variability and reproducibility. This approach aims to determine which factors contribute most strongly to measurement instability and under which conditions performance observations remain sufficiently consistent for reliable comparison and replication.

Accordingly, the central research gap addressed by this study is the lack of a systematic, state-aware evaluation of performance measurement reproducibility in stateful serverless applications. By investigating the relationship between state characteristics and repeated performance measurements, this research seeks to complement existing stateful serverless performance studies and provide a methodological foundation for more reliable and reproducible experimental evaluation. Furthermore, the interaction between state characteristics, workload intensity, and execution-environment conditions has not been sufficiently characterized from a reproducibility perspective. Conventional benchmarking can report average latency, tail latency, throughput, or execution cost, but these measurements alone do not establish whether the reported observations remain stable when the same experiment is repeated.

Therefore, a research gap exists between conventional performance evaluation and state-aware performance reproducibility in serverless systems. There is a need for an experimental framework that treats state-related characteristics as explicit experimental factors and evaluates their effect on the stability of repeated performance measurements.

This research addresses this gap by systematically varying state size, state-access patterns, and workload conditions while repeatedly executing equivalent experimental configurations. The study measures both function-level latency and end-to-end client-observed latency to capture variability at different levels of the execution path. Statistical measures such as standard deviation, coefficient of variation, confidence intervals, and latency distributions will be used to quantify performance variability and reproducibility.

The objective is not to introduce a new state-management or state-placement mechanism, but to provide empirical evidence about how state-related factors influence the reproducibility of performance measurements in stateful serverless applications. The findings are intended to identify the conditions under which performance measurements become unstable and to provide a methodological basis for more reproducible benchmarking of stateful serverless systems.

2.6 Research Objectives

The primary objective of this research is to investigate the reproducibility of performance measurements in stateful serverless applications under controlled and repeated experimental conditions. The study focuses on understanding how state-related characteristics and workload conditions influence performance variability.

The specific objectives of this research are:

1. To investigate the effect of different state sizes on the performance variability of stateful serverless applications.
2. To evaluate how different state-access patterns influence the reproducibility of repeated performance measurements.

3. To analyze the influence of workload conditions on performance variability under different state configurations.
4. To examine the interaction between state characteristics, workload conditions, and execution-environment factors.
5. To quantify performance variability using statistical measures such as mean, standard deviation, coefficient of variation, confidence intervals, and latency distributions.
6. To determine the experimental conditions under which performance measurements remain stable or exhibit significant variability.
7. To develop an experimental methodology for evaluating state-aware performance reproducibility in stateful serverless applications.

2.7 Research Questions

Based on the identified research gap, this study addresses the following research questions:

1. **RQ1:** How does state size affect the variability of performance measurements in stateful serverless applications under repeated and equivalent experimental conditions?
2. **RQ2:** How do different state-access patterns influence the reproducibility of performance measurements in stateful serverless applications?
3. **RQ3:** How does workload intensity interact with state characteristics to influence performance variability and reproducibility?
4. **RQ4:** To what extent do execution-environment conditions contribute to variations in repeated performance measurements of stateful serverless applications?
5. **RQ5:** Which combination of state size, state-access pattern, workload conditions, and execution-environment factors contributes most strongly to performance measurement instability?
6. **RQ6:** Can statistical measures such as standard deviation, coefficient of variation, confidence intervals, and latency distributions provide reliable indicators of performance reproducibility in stateful serverless applications?

The research questions are designed to guide a controlled experimental evaluation in which state-related and workload-related parameters are systematically varied while equivalent configurations are executed repeatedly. The resulting measurements will be analyzed using statistical techniques to determine the stability, variability, and reproducibility of observed performance.

3 Research Methodology

This study adopts an experimental methodology to investigate performance reproducibility in stateful serverless applications. The methodology is designed to evaluate how state-related characteristics and workload conditions influence the variability of performance measurements under repeated experimental conditions.

The experiment uses a stateful serverless application in which application state is maintained across function invocations. The same experimental configuration is executed repeatedly, and performance measurements are collected for each invocation. The collected measurements are then analyzed using statistical techniques to quantify performance variability and reproducibility.

The experimental methodology consists of four major stages: experimental configuration, repeated execution, performance measurement, and statistical analysis. During the experimental configuration stage, the state and workload conditions are defined. During repeated execution, the serverless function is invoked multiple times using the same configuration. Performance metrics are recorded for every invocation. Finally, the collected data are analyzed using descriptive and variability-based statistical measures.

3.1 Experimental Design

The experimental design is structured to evaluate the reproducibility of performance measurements in a stateful serverless environment through repeated executions of equivalent configurations. The experiment maintains persistent application state across function invocations and records performance measurements for each invocation.

Three invocation workloads are considered in the initial experimental evaluation: 10, 20, and 50 invocations. For each workload configuration, the serverless function is invoked repeatedly using the same state identifier. The persistent state is updated during each invocation, allowing the study to observe performance behavior across consecutive state-dependent executions.

For every invocation, the experiment records the invocation number, total number of invocations in the experiment, state identifier, state counter, function execution latency, and end-to-end client-observed latency. These measurements are stored in CSV files for subsequent statistical analysis.

The experimental datasets consist of three configurations, namely `results_10.csv`, `results_20.csv`, and `results_50.csv`. The collected measurements are analyzed to determine the mean, median, standard deviation, minimum, maximum, and coefficient of variation of the observed latency values.

The design enables comparison of performance variability across different numbers of repeated invocations while maintaining the same stateful execution mechanism. This provides an initial basis for evaluating whether performance measurements remain stable as the number of repeated observations increases.

3.2 System Architecture

The experimental system consists of a client-side experiment runner, an AWS Lambda function, and a persistent state store. The architecture is designed to maintain application state across multiple serverless function invocations while collecting performance measurements for each execution.

The experiment runner is implemented in Python and is responsible for invoking the serverless function repeatedly and recording the resulting performance measurements. For each invocation, the runner sends a request containing the state identifier and receives the response generated by the Lambda function.

The AWS Lambda function represents the stateful serverless application under evaluation. During execution, the function accesses the persistent state associated with the provided state identifier and updates the state after each invocation. The state counter is used to verify that state is preserved across consecutive invocations.

Amazon DynamoDB is used as the persistent state storage layer. It maintains the state associated with the application and allows the Lambda function to retrieve and update the state across independent function executions. This persistent storage mechanism enables the experiment to evaluate performance under state-dependent execution rather than treating each invocation as a completely stateless operation.

For every invocation, two latency measurements are collected. The first is the Lambda execution latency, which represents the execution time observed for the serverless function. The second is the client-observed latency, which represents the end-to-end time measured by the experiment runner, including the communication overhead between the client and the serverless service.

The overall architecture can therefore be represented as:

Experiment Runner → AWS Lambda → Amazon DynamoDB

The collected measurements are stored in CSV files and subsequently used for statistical analysis. This architecture provides a controlled environment for studying the relationship between persistent state and the variability of repeated serverless performance measurements.

3.3 Experimental Variables

The experiment defines a set of independent, dependent, and controlled variables to systematically evaluate performance variability in the stateful serverless application.

3.3.1 Independent Variables

The primary experimental variable in the initial evaluation is the number of repeated function invocations. Three invocation configurations are evaluated:

10, 20, and 50 invocations. These configurations provide different sample sizes for examining the stability of observed performance measurements.

In the extended experimental design, state-related characteristics such as state size and state-access pattern can be varied systematically to investigate their influence on performance reproducibility.

3.3.2 Dependent Variables

The primary dependent variables are Lambda execution latency and client-observed latency. Lambda execution latency represents the execution time associated with the serverless function, while client-observed latency captures the end-to-end time measured by the experiment runner.

The variability of these measurements is evaluated using statistical indicators including mean, median, standard deviation, minimum, maximum, and coefficient of variation.

3.3.3 Controlled Variables

To improve the comparability of repeated measurements, the experiment keeps the serverless function, state identifier, AWS region, state-management mechanism, and experimental execution procedure consistent across the initial configurations.

The same stateful serverless application is used for the 10, 20, and 50-invocation experiments. The persistent state is maintained using the same state-management mechanism, and measurements are collected using the same experiment runner and measurement procedure.

Maintaining these factors consistently allows the observed differences in performance variability to be analyzed without intentionally changing the underlying application architecture.

3.4 Experimental Procedure

The experimental procedure consists of a sequence of controlled steps for executing the stateful serverless application and collecting performance measurements.

First, the stateful serverless application is deployed using AWS Lambda, with Amazon DynamoDB used as the persistent state storage layer. A unique state identifier is used to maintain the application state across repeated function invocations.

Second, the Python-based experiment runner is configured to invoke the deployed Lambda function. For each invocation, the experiment runner sends the required state identifier and records the response returned by the serverless function.

Third, three experimental configurations are executed using 10, 20, and 50 repeated invocations. Within each configuration, the same stateful application and experimental procedure are maintained across consecutive invocations.

During each invocation, the experiment records the invocation number, total number of invocations, state identifier, state counter, Lambda execution latency, and client-observed latency. The state counter is also recorded to verify that persistent state is maintained across consecutive executions.

The collected measurements are stored as CSV datasets named `results_10.csv`, `results_20.csv`, and `results_50.csv`. These datasets are subsequently processed using a Python-based analysis script.

Finally, descriptive statistical analysis is performed on the collected latency measurements. Mean, median, standard deviation, minimum, maximum, and coefficient of variation are calculated for both Lambda execution latency and client-observed latency. The resulting statistics are used to evaluate the variability and stability of repeated performance measurements.

3.5 Performance Metrics

The performance of the stateful serverless application is evaluated using multiple latency and variability metrics. These metrics are selected to capture both the execution-level performance of the serverless function and the end-to-end performance observed by the experiment client.

3.5.1 Lambda Execution Latency

Lambda execution latency represents the time required for the serverless function to execute. This metric is used to evaluate variations in the execution performance of the function across repeated invocations.

3.5.2 Client-Observed Latency

Client-observed latency represents the end-to-end time measured by the experiment runner for each function invocation. In addition to function execution, this measurement may include communication and service-level overhead between the experiment runner and the serverless platform.

3.5.3 Mean Latency

Mean latency is calculated to represent the average performance observed across repeated invocations. It provides a central measure for comparing different experimental configurations.

3.5.4 Median Latency

Median latency represents the middle observation after the latency measurements are ordered. It is included to provide a measure that is less affected by unusually high latency observations.

3.5.5 Standard Deviation

Standard deviation is used to quantify the dispersion of latency measurements around their mean. A higher standard deviation indicates greater variation among repeated observations, while a lower value indicates more consistent measurements.

3.5.6 Coefficient of Variation

The coefficient of variation (CV) is used as a normalized measure of performance variability. It relates the standard deviation to the mean and allows variability to be compared across experimental configurations with different average latency values.

3.5.7 Latency Distribution

Latency distributions are examined to identify the spread and occurrence of unusually high or low observations. The distribution provides additional information about performance instability that may not be visible from the mean value alone.

These metrics collectively provide a quantitative basis for evaluating the stability and reproducibility of repeated performance measurements in the stateful serverless environment.

3.6 Statistical Analysis

The collected performance measurements are analyzed using descriptive statistics to quantify the variability of repeated serverless executions. The analysis is performed separately for Lambda execution latency and client-observed latency.

For each experimental configuration, the mean and median latency are calculated to represent the central tendency of the observations. Standard deviation is calculated to quantify the absolute dispersion of the measurements around the mean.

The coefficient of variation (CV) is additionally calculated to provide a normalized measure of variability. It is expressed as a percentage and is calculated as:

$$CV = \frac{\sigma}{\mu} \times 100 \quad (1)$$

where σ represents the standard deviation of the latency measurements and μ represents their mean.

The minimum and maximum latency values are also recorded to identify the observed performance range and potential outlier observations.

The statistical analysis is applied to the three collected experimental datasets, namely `results_10.csv`, `results_20.csv`, and `results_50.csv`. The resulting statistics are stored in `summary_statistics.csv`.

The coefficient of variation is particularly useful for the reproducibility analysis because it allows the relative variability of different experimental configurations to be compared independently of their absolute mean latency. Lower variability indicates greater consistency among repeated measurements, whereas higher variability indicates greater measurement instability.

The statistical results are subsequently used to compare the performance behavior of the different invocation configurations and to identify conditions associated with increased variability in the stateful serverless execution environment.

4 Experimental Results

4.1 Dataset Overview

The experimental evaluation produced three datasets corresponding to 10, 20, and 50 repeated invocations of the stateful serverless function. Each dataset contains the measurements collected during consecutive executions of the same stateful application.

The datasets are stored as `results_10.csv`, `results_20.csv`, and `results_50.csv`. Each record contains the invocation number, total number of invocations, state identifier, state counter, Lambda execution latency, and client-observed latency.

The state counter increases sequentially across the collected invocations, demonstrating that the same persistent state is being accessed and updated during the experiment. The three datasets therefore provide repeated observations of the stateful serverless execution under different numbers of invocations.

The initial dataset contains 10 observations, the second dataset contains 20 observations, and the third dataset contains 50 observations. In total, 80 invocation-level observations were collected for the initial experimental evaluation.

The collected datasets are subsequently analyzed using descriptive statistical measures to identify differences in central tendency and performance variability between the three invocation configurations.

4.2 Latency Results

The measured latency results show differences in the observed performance across the three invocation configurations. Both Lambda execution latency and client-observed latency were analyzed to distinguish function-level execution behavior from end-to-end client-side performance.

For the 10-invocation configuration, the mean Lambda execution latency was 57.16 ms, with a median of 30.07 ms. The relatively large difference between the mean and median is associated with a high-latency observation of 288.05 ms during the first invocation. The mean client-observed latency was 424.02 ms, while the median was 233.27 ms.

For the 20-invocation configuration, the mean Lambda execution latency decreased to 33.56 ms, with a median of 29.03 ms. The mean client-observed latency was 233.08 ms, with a median of 229.58 ms. The smaller difference between the mean and median indicates more consistent observations compared with the 10-invocation configuration.

For the 50-invocation configuration, the mean Lambda execution latency was 32.20 ms, with a median of 30.35 ms. The mean client-observed latency was 230.21 ms, while the median was 224.86 ms. These values are close to those observed in the 20-invocation configuration.

The results indicate that the initial 10-invocation experiment contains a substantially higher latency observation, which increases the overall mean. As the number of observations increases from 20 to 50, the average Lambda execution latency remains relatively stable, changing from 33.56 ms to 32.20 ms. Similarly, client-observed latency remains close to 230 ms in both configurations.

These results suggest that the initial performance measurements can be strongly affected by individual high-latency observations. Therefore, reporting only the average latency may not be sufficient for evaluating performance reproducibility. Variability metrics are considered in the following section to provide a more complete assessment of measurement stability.

4.3 Variability Results

The variability analysis provides additional information about the stability of the measured performance. Standard deviation and coefficient of variation (CV) were used to quantify the dispersion of Lambda execution latency and client-observed latency.

For the 10-invocation configuration, Lambda execution latency had a standard deviation of 81.33 ms and a coefficient of variation of 142.28%. The client-observed latency showed a standard deviation of 601.35 ms and a coefficient of variation of 141.82%. These relatively high variability values indicate substantial dispersion in the initial set of observations.

The 20-invocation configuration showed considerably lower variability. Lambda execution latency had a standard deviation of 8.03 ms and a coefficient of variation of 23.92%. Client-observed latency had a standard deviation of 13.73 ms and a coefficient of variation of 5.89%.

For the 50-invocation configuration, Lambda execution latency had a standard deviation of 6.16 ms and a coefficient of variation of 19.13%. The client-observed latency had a standard deviation of 12.49 ms and a coefficient of variation of 5.43%.

The results demonstrate a substantial reduction in relative variability when moving from the 10-invocation configuration to the larger experimental configurations. In particular, Lambda execution latency decreased from a coefficient of variation of 142.28% for 10 invocations to 23.92% for 20 invocations and 19.13% for 50 invocations.

Client-observed latency shows a similar but less pronounced pattern. Its coefficient of variation decreased from 141.82% for 10 invocations to 5.89% for

20 invocations and 5.43% for 50 invocations.

These observations indicate that individual high-latency measurements can strongly influence the variability of a small sample. Increasing the number of repeated observations provides a more stable characterization of the observed performance. However, these results should be interpreted as an initial reproducibility evaluation, since the present experiment varies the number of observations rather than systematically varying state size or state-access patterns.

4.4 Comparison of 10, 20, and 50 Invocations

A comparison of the three experimental configurations shows that the observed performance variability changes considerably with the number of repeated invocations.

The 10-invocation configuration exhibits the highest variability for both Lambda execution latency and client-observed latency. The coefficient of variation is 142.28% for Lambda execution latency and 141.82% for client-observed latency. This behavior is primarily influenced by the high-latency observation recorded during the initial invocation.

When the number of observations is increased to 20, the coefficient of variation decreases to 23.92% for Lambda execution latency and 5.89% for client-observed latency. The corresponding standard deviations are also substantially lower than those observed in the 10-invocation configuration.

The 50-invocation configuration provides the most stable measurements among the three configurations. Lambda execution latency has a coefficient of variation of 19.13%, while client-observed latency has a coefficient of variation of 5.43%. The mean Lambda execution latency is also similar to that of the 20-invocation configuration, with values of 33.56 ms and 32.20 ms respectively.

Overall, the comparison indicates that increasing the number of repeated observations reduces the influence of individual high-latency observations and provides a more stable estimate of performance variability. However, the results do not imply that a larger number of invocations inherently improves the underlying serverless system performance. Rather, a larger sample provides a more stable basis for characterizing the observed performance behavior.

The comparison also highlights an important consideration for reproducible serverless experimentation: small experimental samples may produce highly variable statistical estimates when occasional high-latency observations are present. Consequently, repeated measurements and explicit variability metrics are important when evaluating the reproducibility of stateful serverless performance.

4.5 Key Findings

The experimental evaluation produces several important findings regarding performance variability and reproducibility in the evaluated stateful serverless application.

1. **High variability in small samples:** The 10-invocation configuration

shows substantially higher variability than the larger configurations. The coefficient of variation reaches 142.28% for Lambda execution latency, indicating that a small number of observations can be strongly affected by individual high-latency executions.

2. **Reduction in variability with repeated measurements:** Increasing the number of observations from 10 to 20 results in a substantial reduction in variability. Lambda execution latency CV decreases from 142.28% to 23.92%, while client-observed latency CV decreases from 141.82% to 5.89%.
3. **Greater stability with 50 observations:** The 50-invocation configuration provides the lowest observed variability among the evaluated configurations. Lambda execution latency has a CV of 19.13%, while client-observed latency has a CV of 5.43%.
4. **Stable average latency after the initial sample:** The mean Lambda execution latency changes only slightly between the 20-invocation and 50-invocation configurations, from 33.56 ms to 32.20 ms. This suggests that increasing the sample size beyond 20 observations has a smaller effect on the estimated average latency than the transition from 10 to 20 observations.
5. **Importance of variability-aware evaluation:** The results demonstrate that mean latency alone may not adequately describe the stability of serverless performance measurements. Variability metrics such as standard deviation and coefficient of variation provide additional information that is important when assessing performance reproducibility.

Overall, the findings indicate that repeated measurements are important for obtaining stable performance estimates in stateful serverless experiments. The results also motivate a more systematic investigation of state-related factors, including state size and state-access patterns, in future experimental configurations.

5 Discussion

The experimental results provide initial evidence that performance measurements in stateful serverless environments can exhibit substantial variability across repeated executions. The observed differences between the 10-, 20-, and 50-invocation configurations demonstrate that the number of repeated observations can influence the stability of statistical measurements.

The most significant observation is the high variability observed in the 10-invocation configuration. The presence of a single high-latency observation substantially increases both the mean and standard deviation. This demonstrates that small experimental samples may provide an unstable representation of serverless performance when occasional high-latency executions occur.

As the number of observations increases, the measured variability becomes lower. The 20- and 50-invocation configurations produce considerably lower coefficients of variation than the 10-invocation configuration. This suggests that repeated measurements are important for obtaining a more stable characterization of serverless performance.

However, the results should not be interpreted as evidence that increasing the number of invocations directly improves the performance of the serverless application. Instead, the larger sample sizes provide more observations from which the underlying performance behavior can be characterized. This distinction is important when designing reproducible serverless experiments.

The results also demonstrate the importance of reporting variability in addition to average latency. For example, the 20- and 50-invocation configurations have relatively similar mean Lambda execution latencies, but their variability measures provide additional information about the stability of the observations. Therefore, reproducibility analysis should consider both central-tendency and dispersion-based metrics.

From the perspective of the proposed research gap, the current experiment provides an initial baseline for studying state-aware performance reproducibility. The experiment confirms that repeated measurements can vary even when the same stateful serverless function is executed under a controlled configuration. However, the present evaluation does not yet isolate the effects of different state sizes or state-access patterns.

Therefore, the next stage of the study should systematically vary these state-related factors while maintaining comparable experimental conditions. Such an evaluation can help determine whether changes in state characteristics are associated with measurable differences in performance variability and reproducibility.

6 Threats to Validity

Although the experimental results provide useful evidence regarding performance variability in the evaluated stateful serverless application, several factors may affect the generalizability and interpretation of the results.

6.1 Internal Validity

The experiment was conducted using a single stateful serverless function and a single experimental state identifier. Consequently, the observed variability may be influenced by characteristics specific to the implemented function, its state-management mechanism, or its interaction with the selected storage service.

The experiments were also conducted in a cloud execution environment where the underlying infrastructure is managed by the cloud provider. Factors such as resource contention, network conditions, scheduling behavior, and runtime initialization may introduce variability that cannot be fully controlled by the experiment.

6.2 External Validity

The current evaluation uses a limited set of invocation configurations, namely 10, 20, and 50 repeated invocations. Therefore, the observed behavior may not generalize to substantially larger workloads or different workload patterns.

Furthermore, the current experiment does not yet systematically vary state size and state-access patterns. As a result, the present findings should not be interpreted as evidence that a particular state size or access pattern causes a specific level of performance variability.

6.3 Measurement Validity

Two latency measurements are considered in the experiment: Lambda execution latency and client-observed latency. Client-observed latency includes additional overhead outside the function execution itself, such as network communication and client-side processing. Therefore, the two metrics represent different aspects of system performance and should not be treated as equivalent measurements.

The presence of occasional high-latency observations can also have a strong effect on mean-based statistics, particularly when the number of observations is small. For this reason, median, standard deviation, coefficient of variation, and latency ranges are considered alongside the mean.

6.4 Reproducibility Threats

Cloud-based experiments may be affected by transient environmental conditions that are difficult to reproduce exactly across different execution periods. Although the experimental procedure is kept consistent, the underlying cloud infrastructure may change between runs.

To reduce these threats, the experimental configuration, invocation procedure, state identifier, collected measurements, and statistical analysis procedure are explicitly documented. Future experiments will further reduce these threats by repeating each state configuration under controlled workload and execution conditions.

7 Conclusion and Future Work

7.1 Conclusion

This study investigated performance variability and reproducibility in a stateful serverless application through repeated cloud-based experiments. The evaluation considered Lambda execution latency and client-observed latency across configurations containing 10, 20, and 50 repeated invocations.

The experimental results demonstrate that serverless performance measurements can exhibit substantial variability, particularly when the number of observations is small. The 10-invocation configuration showed high variability,

with a coefficient of variation of 142.28% for Lambda execution latency. In contrast, the coefficient of variation decreased to 23.92% and 19.13% for the 20- and 50-invocation configurations, respectively.

Client-observed latency showed a similar pattern, with the coefficient of variation decreasing from 141.82% for 10 invocations to 5.89% for 20 invocations and 5.43% for 50 invocations.

These results indicate that repeated measurements and variability-aware statistical analysis are important when evaluating the reproducibility of stateful serverless performance. Mean latency alone may not adequately represent the stability of an experimental measurement, particularly when individual high-latency observations are present.

The current study therefore provides an initial experimental baseline for state-aware performance reproducibility. However, the present results do not establish the independent effect of state size or state-access patterns. These factors require systematic experimental variation in the next phase of the research.

7.2 Future Work

Future work will extend the current experiment by systematically varying the characteristics of application state and workload conditions.

First, different state sizes will be evaluated to determine whether the amount of persistent state influences latency variability and the reproducibility of repeated measurements.

Second, different state-access patterns will be investigated, including repeated reads, repeated writes, mixed read-write operations, and varying access frequencies. This will allow the study to determine whether the way in which state is accessed contributes to performance instability.

Third, workload intensity will be varied to examine the interaction between workload conditions and state-related characteristics. Different invocation rates and workload levels can be used to determine whether performance variability changes under increasing execution pressure.

Fourth, selected execution-environment factors will be considered to evaluate whether environmental conditions interact with state characteristics and contribute to measurement variability.

Finally, future experiments will use repeated trials for each experimental configuration and apply additional statistical techniques, including confidence intervals and latency-distribution analysis. The resulting dataset will be used to identify relationships between state characteristics, workload conditions, execution environment, and performance reproducibility.

The ultimate objective is to develop a systematic methodology for evaluating and reporting reproducible performance measurements in stateful serverless applications.

8 Experimental Configuration

The experiments were conducted using a stateful serverless application deployed in a cloud environment. The application maintains persistent state using a counter-based state identifier and returns the updated state value after each invocation.

Each experimental configuration consisted of repeated invocations of the same serverless function using the state identifier `test-state`. For each invocation, the experiment recorded both server-side execution latency and client-observed latency.

Three initial invocation configurations were evaluated:

- 10 repeated invocations
- 20 repeated invocations
- 50 repeated invocations

For each invocation, the following variables were recorded:

- Invocation number
- Total number of invocations in the configuration
- State identifier
- State counter value
- Lambda execution latency in milliseconds
- Client-observed latency in milliseconds

The collected observations were stored as CSV files for subsequent statistical analysis. The analysis calculated the mean, median, standard deviation, minimum, maximum, and coefficient of variation for both latency measurements.

The coefficient of variation was used as a normalized measure of performance variability because it relates the standard deviation to the mean. This enables variability to be compared across experimental configurations with different average latency values.

The experimental data consisted of the files `results_10.csv`, `results_20.csv`, and `results_50.csv`. The statistical summary was stored in `summary_statistics.csv`.

The experimental implementation and analysis scripts are maintained separately from the research manuscript to support repeatability of the experimental procedure.

9 Appendix

9.1 Experiment Execution Procedure

The following procedure was used to execute the stateful serverless experiments:

1. Configure the cloud environment and authentication credentials.
2. Deploy the stateful serverless function and configure its persistent state mechanism.
3. Verify that the deployed function can be invoked successfully.
4. Execute the experiment using the state identifier `test-state`.
5. Record the state counter, Lambda execution latency, and client-observed latency for every invocation.
6. Store the collected measurements in CSV format.
7. Repeat the procedure for 10, 20, and 50 invocation configurations.
8. Calculate descriptive statistics for each configuration, including mean, median, standard deviation, minimum, maximum, and coefficient of variation.

9.2 Recorded Data Format

Each experimental observation contains the following fields:

```
invocation
total_invocations
state_id
counter
latency_ms
client_latency_ms
```

The resulting datasets are stored as:

```
data/results_10.csv
data/results_20.csv
data/results_50.csv
```

The statistical analysis produces:

```
analysis/summary_statistics.csv
```

9.3 Reproducibility Procedure

To support reproducibility, the experiment records the experimental configuration and raw observations separately from the statistical analysis. The same invocation procedure can therefore be repeated with different state configurations, workload conditions, and experimental parameters.

Future experimental runs will extend this procedure by varying state size, state-access pattern, and workload intensity while preserving the same measurement and analysis methodology.

10 Results and Discussion

10.1 Experimental Results

The experiments were conducted to investigate the reproducibility and variability of performance measurements in a stateful serverless application. Three experimental dimensions were considered: workload intensity, state size, and state-access pattern. Each configuration was executed repeatedly and performance was evaluated using mean latency, median latency, standard deviation, minimum latency, maximum latency, and coefficient of variation (CV).

The coefficient of variation was used as a normalized measure of performance variability and was calculated as:

$$CV = \frac{\sigma}{\mu} \times 100$$

where σ represents the standard deviation and μ represents the mean latency. A higher CV indicates greater relative variability and therefore lower measurement stability.

10.2 Workload Intensity

The workload experiment considered 10, 20, and 50 repeated invocations. The results show a substantial difference in variability between the three workload configurations.

For 10 invocations, the observed Lambda-side latency had a mean of 57.16 ms and a coefficient of variation of 142.28%. The corresponding client-observed latency had a mean of 424.02 ms and a CV of 141.82%. The high CV indicates substantial measurement variability, which was strongly influenced by unusually high latency observations.

For 20 invocations, Lambda-side latency showed a mean of 33.56 ms with a CV of 23.92%, while client-observed latency had a mean of 233.08 ms and a CV of 5.89%.

For 50 invocations, the Lambda-side latency mean was 32.20 ms with a CV of 19.13%, while client-observed latency had a mean of 230.21 ms and a CV of 5.43%.

These results indicate that a small number of repeated invocations can produce unstable estimates of serverless performance. Increasing the number of repetitions from 10 to 20 substantially reduced the observed variability. The difference between the 20- and 50-invocation experiments was smaller, suggesting that repeated execution provides a more stable estimate of performance.

10.3 State Size

The state-size experiment evaluated three state-size configurations: 100 bytes, 1,000 bytes, and 10,000 bytes. The client-observed latency was used for this comparison because the Lambda-side latency field was not consistently captured in the state-size experiment output.

For a state size of 100 bytes, the mean client-observed latency was 454.88 ms with a standard deviation of 336.14 ms and a CV of 73.90%. The maximum observed latency was 1451.10 ms.

For a state size of 1,000 bytes, the mean client-observed latency decreased to 336.92 ms, with a standard deviation of 109.14 ms and a CV of 32.39%.

For a state size of 10,000 bytes, the mean client-observed latency was 367.29 ms with a standard deviation of 217.59 ms and a CV of 59.24%.

The results do not show a simple monotonic relationship between state size and latency variability. The 1,000-byte configuration produced the lowest observed CV, whereas both the 100-byte and 10,000-byte configurations showed substantially higher variability. This suggests that state size alone may not completely explain performance instability and that other factors, such as execution-environment conditions and state-access behavior, may also contribute to measurement variability.

Therefore, the state-size experiment provides evidence that changing the amount of state can be associated with changes in observed performance stability, but the relationship requires further controlled experiments before a causal conclusion can be made.

10.4 State-Access Patterns

The access-pattern experiment evaluated three state-access behaviors: read-heavy, write-heavy, and mixed access.

The read-heavy configuration showed a Lambda-side mean latency of 46.22 ms and a CV of 119.41%. Its client-observed latency had a mean of 396.50 ms and a CV of 103.16%. These high CV values indicate substantial variability across repeated executions.

The write-heavy configuration showed a lower Lambda-side mean latency of 33.18 ms and a CV of 15.37%. Client-observed latency had a mean of 251.26 ms and a CV of 9.83%.

The mixed access configuration produced a Lambda-side mean latency of 37.30 ms and a CV of 28.73%. Its client-observed latency had a mean of 275.89 ms and a CV of 11.96%.

Among the evaluated access patterns, the read-heavy configuration exhibited the greatest performance variability. In contrast, the write-heavy configuration showed substantially lower variability. The mixed configuration was between the two in terms of observed variability.

These results suggest that state-access behavior can influence the stability of performance measurements. In particular, repeated read operations may introduce greater variability than the write-heavy configuration used in this experiment. However, the results should be interpreted as experimental observations rather than definitive causal evidence because other environmental factors were not fully controlled.

10.5 Discussion of Reproducibility

The combined experimental results provide evidence that repeated performance measurements in stateful serverless applications are not necessarily identical even when the logical experiment remains unchanged.

The workload experiment demonstrates that insufficient repetition can result in highly variable measurements. The state-size experiment shows that different state configurations can exhibit different levels of measurement variability. The access-pattern experiment further indicates that the way state is accessed can influence observed performance stability.

An important observation is the difference between Lambda-side latency and client-observed latency. Client-observed latency includes additional components associated with communication between the experiment client and the serverless platform. Consequently, these two measurements should not be treated as identical performance indicators. Reporting both measurements provides a more complete view of the experimental behavior.

Overall, the findings support the central motivation of this study: performance benchmarking of stateful serverless applications should consider not only average latency but also the variability and reproducibility of repeated measurements. Reporting mean values alone can hide substantial instability, particularly in configurations with high coefficients of variation.

10.6 Answer to Research Questions

RQ1: How does state size influence the variability of performance measurements in stateful serverless applications?

The experiments indicate that state size is associated with differences in observed performance variability. The 1,000-byte configuration showed the lowest client-observed CV (32.39%), while the 100-byte and 10,000-byte configurations showed higher CV values of 73.90% and 59.24%, respectively. However, the results do not establish a monotonic relationship between state size and variability.

RQ2: How do different state-access patterns affect performance reproducibility?

The results indicate that access pattern has a noticeable relationship with measurement variability. The read-heavy configuration produced substantially higher variability than the write-heavy and mixed configurations. The client-observed CV was 103.16% for read-heavy access, compared with 9.83% for write-heavy access and 11.96% for mixed access.

RQ3: How does workload intensity affect the stability of repeated performance measurements?

Increasing the number of repeated invocations generally improved the stability of the measured results. The client-observed CV decreased from 141.82% for 10 invocations to 5.89% for 20 invocations and 5.43% for 50 invocations. This indicates that repeated measurements are important for obtaining more stable performance estimates.

10.7 Summary of Findings

The experiments demonstrate that performance reproducibility in stateful serverless systems is influenced by multiple experimental factors rather than by a single parameter. Workload repetition, state size, and state-access pattern were all associated with differences in observed variability.

The most pronounced variability was observed in the 10-invocation workload and read-heavy access-pattern experiments. In contrast, the 20- and 50-invocation workloads produced considerably more stable client-observed measurements.

These findings support the need for state-aware experimental methodologies in serverless performance research. A reproducibility analysis should therefore report not only central tendency measures such as mean and median latency, but also dispersion measures such as standard deviation and coefficient of variation.

11 Conclusion

This study investigated the reproducibility of performance measurements in stateful serverless applications by systematically varying workload intensity, state size, and state-access patterns. Unlike conventional benchmarking approaches that primarily report average latency, this study focused on the variability and stability of repeated measurements using statistical indicators such as mean, median, standard deviation, and coefficient of variation.

The experimental results demonstrate that repeated executions of the same logical configuration can produce noticeably different performance measurements. The workload experiment showed that increasing the number of repetitions generally resulted in more stable client-observed measurements. The access-pattern experiment showed substantial differences in variability between read-heavy, write-heavy, and mixed state-access behaviors. The state-size experiment also demonstrated that different state configurations can exhibit different levels of observed variability, although the relationship was not monotonic.

The findings indicate that performance reproducibility in stateful serverless systems cannot be adequately characterized using average latency alone. Measurements should also account for dispersion and relative variability because a low average latency may coexist with substantial instability across repeated executions.

The study therefore highlights the importance of treating state-related characteristics as explicit experimental factors when evaluating stateful serverless systems. The proposed state-aware evaluation approach provides a basis for understanding how state configuration, workload conditions, and access behavior contribute to performance measurement variability.

Overall, the results support the need for reproducibility-oriented benchmarking methodologies for stateful serverless applications. Future studies can extend this work by increasing the number of repetitions, evaluating additional state sizes and workloads, controlling execution-environment conditions more strictly, and applying statistical significance testing and multi-factor experimental models to quantify interactions between state characteristics and environmental factors.

12 Limitations and Future Work

12.1 Limitations

This study has several limitations. First, the experiments were conducted using a limited set of state sizes, workload intensities, and state-access patterns. Therefore, the observed results may not represent the complete performance behavior of all stateful serverless applications.

Second, serverless execution environments are influenced by platform-level conditions that are not completely controllable by the experiment. Factors such as resource availability, network conditions, platform scheduling, and cold-start behavior may contribute to observed performance variability.

Third, the current experiments use a relatively small number of repeated executions for each configuration. Although the results demonstrate measurable variability, a larger number of repetitions would provide stronger statistical evidence and more reliable estimates of performance distributions.

Fourth, the study currently considers client-observed latency separately from Lambda-side execution latency. These measurements represent different parts of the execution path and therefore should not be interpreted as equivalent metrics.

Finally, the current analysis primarily uses descriptive statistical measures such as mean, median, standard deviation, and coefficient of variation. More advanced statistical methods could provide stronger evidence regarding the significance of differences between experimental configurations.

12.2 Future Work

Future work will extend the experimental methodology by increasing the number of repetitions for each configuration and evaluating a broader range of state sizes and workload conditions. Additional state-access patterns can also be investigated to determine whether the observed variability is consistent across different state interaction behaviors.

A further direction is to explicitly investigate execution-environment effects, including cold starts, warm starts, resource allocation, invocation timing, and network conditions. Controlling or recording these factors would make it possible to separate state-related variability from platform-related variability.

Future experiments can also employ statistical hypothesis testing and multi-factor experimental designs to quantify the individual and interaction effects of state size, access pattern, workload intensity, and execution environment.

Another important direction is to evaluate the methodology across multiple serverless platforms and state-management technologies. Comparing different cloud providers and persistent-state mechanisms would help determine whether the observed reproducibility behavior is platform-specific or represents a broader characteristic of stateful serverless systems.

Finally, future work will investigate reproducibility thresholds that can be used to determine when a performance measurement is sufficiently stable to support reliable benchmarking conclusions. Such thresholds could contribute toward the development of standardized state-aware benchmarking methodologies for stateful serverless applications.

13 References

References

- [1] J. Scheuner and P. Leitner, “Function-as-a-Service performance evaluation: A multivocal literature review,” *Journal of Systems and Software*, vol. 170, Art. no. 110708, 2020, doi: 10.1016/j.jss.2020.110708.
- [2] M. Shahrad, R. Fonseca, I. Goiri, G. I. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, “Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider,” in *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pp. 205–218, 2020.
- [3] S. Eismann, D. E. Costa, L. Liao, C.-P. Bezemer, W. Shang, A. van Hoorn, and S. Kounev, “A case study on the stability of performance tests for serverless applications,” *Journal of Systems and Software*, vol. 202, Art. no. 111294, 2022, doi: 10.1016/j.jss.2022.111294.
- [4] J. Wen, Z. Chen, F. Sarro, and S. Wang, “Unveiling Overlooked Performance Variance in Serverless Computing,” *Empirical Software Engineering*, 2023.

- [5] J. Scheuner, S. Eismann, S. Talluri, E. van Eyk, C. Abad, P. Leitner, and A. Iosup, “Let’s Trace It: Fine-Grained Serverless Benchmarking using Synchronous and Asynchronous Orchestrated Applications,” arXiv preprint arXiv:2205.07696, 2022.
- [6] D. Giantsidi, E. Giortamis, N. Tornow, F. Dinu, and P. Bhatotia, “FlexLog: A Shared Log for Stateful Serverless Computing,” in *Proceedings*, 2023.
- [7] “Transactions across Serverless Functions Leveraging Stateful Dataflows,” *Information Systems*, vol. 108, Art. no. 102015, 2022, doi: 10.1016/j.is.2022.102015.
- [8] M. Wiesholler, F. Dinu, J. Picorel, and P. Bhatotia, “IndiLog: Bridging Scalability and Performance in Stateful Serverless Computing with Shared Logs,” in *Proceedings of the 17th ACM International Systems and Storage Conference (SYSTOR)*, 2024.
- [9] K. Psarakis, G. Christodoulou, G. Siachamis, M. Fragkoulis, and A. Katsifodimos, “State Migration in Styx: Towards Serverless Transactional Functions,” *The VLDB Journal*, vol. 35, no. 4, Art. no. 31, 2026, doi: 10.1007/s00778-026-00971-x.
- [10] S. Eismann, J. Scheuner, E. van Eyk, M. Schwinger, J. Grohmann, N. Herbst, C. L. Abad, and A. Iosup, “A Review of Serverless Use Cases and their Characteristics,” SPEC Research Group Technical Report, SPEC-RG-2020-5, 2020.